

A I P O C K E T A G E N C Y

aipocketagency.com

[apa · kit 01]

The Dispatch Playbook

The operator manual for running parallel Claude Code agents without them stepping on each other. Eleven sections — worktrees, standing rules, verification discipline, the named failure modes, the recovery sequences.

KIT

01 / 05

PRICE

\$15 USD

FORMAT

14 pages

TABLE OF CONTENTS

01	Why Dispatch matters	p.2
02	The rogue-agent fear and why it's solvable	p.3
03	The 3-tier task model: cowork vs code vs main thread	p.4
04	Worktree-based parallel lane pattern	p.5
05	Standing rules every lane needs	p.7
06	How to write a clean lane prompt	p.9
07	Verification discipline: four layers of done	p.10
08	Parallel orchestration patterns	p.11
09	Common failure modes	p.12
10	Real example walkthroughs	p.13
11	What to do when it goes wrong	p.14

SECTION 1.

Why Dispatch matters

Single-thread Claude Code is one conversation, one context window, one agent doing one thing at a time. When the context fills up — and it always fills up — you start over. That's the wall.

You can be productive inside one chat for a couple hours. After that the agent starts losing track of what it already decided, you start re-pasting paragraphs from earlier, the work slows down, you hit auto-compaction and lose the thread anyway. Single-thread works fine for tight, contained tasks. It breaks the moment your day involves more than one moving piece.

Dispatch is the orchestrator that spawns sub-agents in parallel. Each sub-agent has its own context window, focused on one task. You stay in the main thread. They go off and work. They report back. You decide what to do with the result.

The actual unlock isn't raw speed. It's that you stop *watching* one agent. Watching a single agent type for ten minutes is the worst kind of work — too engaged to do anything else, not engaged enough to feel like you're producing. Spawn four lanes, each one finishes on its own timeline, you check back when they have something to show. **That's the difference between babysitting and orchestrating.**

Concrete before-and-after. Last week I had four things to ship on the same Next.js marketing site: rewrite the hero copy, fix a broken footer URL, remove a stale Twitter link, and add a monospace marker to the manifesto headline. Single-thread version of that day would have been me opening Claude Code, doing thing one, waiting, reviewing, doing thing two, waiting, reviewing — about three hours of back-and-forth in one chat that filled up halfway through and forced me to start over. Dispatch version: I spawned four code tasks in parallel, gave each one a verbatim instruction and its own worktree, and went and made coffee. All four reported back inside an hour. Each one had pushed its own commit to main. I verified, shipped, moved on.

The cost of Dispatch is complexity. Now you're managing four agents instead of one. Four agents means four sets of instructions, four worktrees, four pushes to verify, four chances something goes sideways. That complexity is the reason most operators who hear about Dispatch try it once, get scared by a merge conflict, and go back to single-thread.

The rest of this playbook is the pattern for handling the complexity. Not avoiding it — handling it. Once the pattern is in place, the four-lane day is the normal day, and the single-thread day is what you go back to only when the work doesn't actually split.

SECTION 2.

The rogue-agent fear and why it's solvable

Four agents are running at the same time. They all have write access. You can't watch every keystroke. The fear isn't crazy. I had it too — the first time I ran four lanes I sat there refreshing GitHub like a maniac.

The fix is to reframe what the agents are actually doing. They aren't random. They follow the instructions you write. If you write tight instructions — verbatim, scoped, with explicit “don't touch these files” guardrails — they stay in their lane. If you write loose instructions — “fix the marketing site” — they do whatever they think you meant, which is the real source of the fear.

So your job shifts. You're not watching one agent type. You're writing tighter instructions and verifying what came back. The work moves to the front (the prompt) and the back (the verify). The middle (the typing) is the part you give up watching.

Three guardrails make this safe in practice.

The first is **worktrees**. Each lane gets its own working directory pointing at the same repo, on its own branch. Lane A and Lane B both editing `src/page.tsx` doesn't blow up because they're not editing the same file on disk — they each have a copy. Section 4 unpacks this.

The second is **standing rules in** `CLAUDE.md` **and** `AGENTS.md`. Every lane reads those files at session start. The rules cover commit conventions, where secrets live, which patterns to follow, what's off-limits. You write the rules once. Every future lane inherits them. Section 5 unpacks this.

The third is **verification discipline**. You do not trust a lane's self-report. “Pushed to main” is what the lane says it did. `git log origin/main --oneline` is what actually happened. The gap between those two has bitten me more times than I want to admit, and the discipline of checking is the difference between a clean operation and a quiet four-hour disaster where you thought five things shipped and zero actually did. Section 7 unpacks the verify discipline in full.

The fear isn't unjustified. Don't take this as “don't be careful.” Take it as “the carefulness has a structure.” Once the structure is in place, you spawn four lanes the same way you'd send four emails — quick, clear asks, check back later.

One more thing worth saying out loud: the fear is loudest the first time. After the first clean four-lane day where every lane reports back with a verified SHA and you ship four things before lunch, the fear becomes a habit of verification instead of a knot in your stomach. It

doesn't fully go away — and it shouldn't — but it stops being the thing that keeps you in single-thread.

SECTION 3.

The 3-tier task model: cowork vs code vs main thread

Before you spawn anything, you decide which kind of agent you're spawning. There are three tiers. Most of the friction operators hit with Dispatch comes from picking the wrong tier for the work.

Main thread is your conversation. You are here. You orchestrate, you decide, you review. Don't try to make the main thread do the lane work — its job is to manage the lanes, not be one. The temptation in the early days is to "just do it here while we're talking." Resist. The main thread is where you read reports and make calls. The moment you start editing files in the main thread instead of spawning a lane for them, you've lost the orchestrator vantage point and you're back to single-thread.

Cowork task is a sandboxed sub-agent. It runs in its own context, reads your filesystem via FUSE (read-only by default), and writes to a sandbox volume that doesn't touch your real repo. Cowork is what you want for research, web fetches, document analysis, drafting markdown, summarizing transcripts — anything that doesn't need to push code. Cowork is faster to spawn, cheaper to run, and structurally incapable of breaking your repo because it can't write to it.

Code task is a sub-agent that runs on the host machine in a real git worktree. Real filesystem, real git, real `bun install`, real builds. This is the tier you use for actual code changes — commits, pushes, edge function deploys, anything that has to interact with your toolchain. It's slower to spawn (creating a worktree and checking out a branch costs a few seconds), but it's the only tier that can ship code.

The choice rule is short: **if it touches `git` or `node_modules`, use a code task. If it just reads, researches, or writes a markdown file, use cowork.**

Worked examples

RESEARCH A SECTION	cowork — file reads + markdown out
REWRITE + PUSH HERO COPY	code task — touches repo, has to push
COMPETITOR RESEARCH DOC	cowork — web fetches + markdown
ADD + DEPLOY EDGE FUNCTION	code task — touches repo + deploy

SUMMARIZE CALL cowork — read + write markdown
TRANSCRIPTS

BUMP DEP + RUN code task — touches `node\_modules` + build
BUILD

The most common mistake is spawning a code task when cowork would do. Code tasks are heavier and slower, and they take a worktree slot you'll have to clean up later. If a research lane and a code lane both want to run at the same time, the cowork lane will spawn faster and finish faster. Default to cowork unless the work genuinely requires git or a build.

The less common but more expensive mistake is doing code work in cowork. Cowork can't push to your repo. If you accidentally hand a "rewrite the homepage and ship it" task to a cowork sandbox, the cowork agent will dutifully edit files in the sandbox volume, report success, and you'll find out an hour later that nothing landed because there was no host-side git to push from. The agent isn't lying — it really did edit the files. They just edited files that don't exist where your repo lives.

One side note worth tracking: the FUSE shim that cowork uses to read your filesystem is also the reason cowork can't run most modern bundlers locally. Bundlers thrash temp files, FUSE blocks the `unlink()` calls, and the build dies with `EPERM unlink`. This isn't a bug to work around — it's a signal that you're in the wrong tier. If a research lane suddenly needs to run a build, that's the moment to spawn a code task instead.

SECTION 4.

Worktree-based parallel lane pattern

A git worktree is a second working directory pointing at the same repo, on its own branch. You can have as many worktrees as you want, all sharing the same `.git` history. Worktrees are the reason parallel lanes don't step on each other.

The setup is automatic if you're using Dispatch's code tasks. Each one creates a worktree at `<repo>/.claude/worktrees/<adjective-name>` on a branch like `task/<adjective-name>`. The lane agent `cd` s into that path, edits files there, runs builds there, commits there, pushes from there. Your main checkout — wherever you opened the repo from — is untouched. You can have your editor open in the main checkout, watch a deploy log in your terminal, and four agents running in worktrees beside you, all on the same repo, none of them aware of the others.

Why this matters: Lane A editing `src/app/page.tsx` and Lane B also editing `src/app/page.tsx` does not blow up. They each have their own copy. They edit independently, they commit independently, and they push independently. The merges happen one at a time on `origin/main` as each lane finishes its work.

Concrete example — four lanes, one Next.js site

- › Lane 1: rewrite the hero copy block on `src/app/page.tsx`
- › Lane 2: fix a broken URL in `src/components/Footer.tsx`
- › Lane 3: remove a stale Twitter link in `src/components/SocialLinks.tsx`
- › Lane 4: add a monospace marker to a heading in `src/app/manifesto/page.tsx`

Each lane gets its own worktree directory and its own branch. Each lane reads whatever it needs to read — including files outside its target if the import graph touches them — without anyone clobbering anyone. Each lane commits to its branch, then pushes its commit to `origin/main`:

```
git push origin HEAD:main
```

They land sequentially because git serializes pushes to main on the remote. First one in wins, second one fast-forwards, third one rebases, fourth one rebases. No conflicts because they touched different files. The whole flight lands within seconds of each other and you've shipped four commits while you were checking email.

Two-lane same-file workaround

The downside surfaces when two lanes touch the **same file**, in particular the **same lines** of the same file. At that point you get a merge conflict at push time and one lane has to rebase against the other's commit. There are two ways to avoid this:

1. **Scope lanes to disjoint files.** Easiest version of the pattern. If you're shipping four edits and they're all in different files, you're safe. Most multi-lane days look like this.
2. **If they have to share a file, sequence them.** Lane A goes first. After it lands on main, Lane B reads the new state of the file and rebases its work onto it. Slower than full parallel, still faster than single-thread, and dramatically less painful than untangling two conflicting commits after the fact.

Shared-state risks worth naming

A shared concern that bites people more than they expect: **shared library files**. If you have a `types.ts` or a generated `database.types.ts` and two lanes both regenerate it, you will have a merge conflict every time. Either pick one lane to own the regeneration step, or do it manually in the main thread after the lanes ship. The same goes for lockfiles — two lanes that both `bun install` will both update `bun.lockb`, and one will lose at push time.

Migrations have a similar shared-state risk if your stack uses timestamp-prefixed migration files. Two lanes both creating `20260512000001_*.sql` will collide. The fix is to assign each lane its own timestamp prefix in the prompt: Lane A gets `20260514*`, Lane B gets `20260515*`, Lane C gets `20260516*`. No two lanes touch the same prefix. No collision.

One last note on worktrees themselves: they accumulate. Every code task you spawn leaves a directory and a branch. If you don't clean them up, you'll wake up next week with forty stale worktrees and a `git branch -a` output that takes a paragraph to scroll. Rule 3 in the next section is the cleanup discipline.

SECTION 5.

Standing rules every lane needs

Every lane needs to read the same rulebook before it starts. The rulebook lives in `CLAUDE.md` and `AGENTS.md` at the root of your repo. Five rules are non-negotiable for parallel lane work.

These five rules go in `CLAUDE.md`. They also get repeated in every lane's task prompt as a session-hygiene preamble, because lane agents read their prompt with more attention than they read the standing file. They are the difference between a clean four-lane day and a Friday afternoon spent untangling someone else's mess.

Rule 1 — Rebase on origin/main at start

Before the lane reads any feature files or starts editing, it runs:

```
git fetch origin
git rebase origin/main
```

If the rebase conflicts, the lane stops and reports. It does not attempt to resolve. A conflict at session start means main moved in a way that clashes with the worktree's base — that's an orchestration issue for the main thread to handle, not something the lane should silently paper over.

A worktree branch can sit idle for hours while you spawn other work. By the time you come back to it, main has moved on. A lane that starts editing without rebasing is editing against a stale base. Rebase first, edit second.

Rule 2 — Verify the push actually landed on origin/main

A lane that says "pushed" is not the same as a lane that pushed to `origin/main`. A lane operating inside a worktree branch can run `git push` and successfully push to its worktree branch — never reaching main — while truthfully reporting "pushed."

Three things make this rule enforceable:

1. The lane uses `git push origin HEAD:main` explicitly, not just `git push`. The explicit refspec forces the push to land on main rather than on whatever branch the local HEAD happens to be tracking.
2. After the push, the lane runs `git log origin/main -1 --format='%H %s'` and includes the resulting SHA and subject line in its report.

3. The main thread independently verifies with `git ls-remote origin main` before declaring anything shipped.

If CI runs against main, the lane also waits on `gh run watch --exit-status` for the run triggered by its push. “Pushed and CI green” is the only acceptable definition of shipped.

Rule 3 — Self-cleanup after a verified push

Once the push is verified, the lane cleans up after itself:

```
cd <main repo path>
git worktree remove <worktree path>
git push origin --delete <branch-name>
git branch -D <branch-name>
```

This is non-negotiable. Branches and worktrees accumulate faster than anyone manually cleans them. A scheduled cleanup task can sweep abandoned branches every few hours as a backstop, but the primary enforcement is every lane cleans up its own mess. The backstop is for failures, not for normal operations.

Rule 4 — Build before declaring success

A green commit is not proof the build works. A passing local unit test is not proof the build works. The build proves the build works.

For the kind of frontend repos most of this playbook applies to, that means the lane runs `bun run build` and waits for an exit code zero before pushing. If you use Vercel or a similar deploy target, “the build worked locally” is the floor and rule 2’s CI check is the ceiling.

A specific trap: if you’re working in a sandboxed cowork environment, the local build can fail with `EPERM unlink` errors because FUSE doesn’t allow the temp-file thrash modern bundlers do. That isn’t a bug to fight — it’s a signal that you spawned a cowork task when you needed a code task. Move the build to a code task on the host filesystem.

The other trap, for repos with edge functions or deploy-separate artifacts: a green build and a green push do not deploy your edge functions. Supabase edge functions, Vercel functions with separate deploy steps, Cloudflare workers — all need an explicit deploy command after the push. The lane is responsible for running that deploy and reporting the deploy timestamp.

Rule 5 — Cite the SHA, not just “pushed”

When the lane reports back, demand the actual commit SHA on origin/main. Not “pushed.” Not “shipped.” The seven-character SHA from `git log origin/main -1 --format='%h %s'`. Compose the report like this so it’s scannable:

```
Lane: rewrite-hero
Branch: task/swift-tesla-9a2c
```

```
Commit SHA: a7f3e21  
Origin/main tip: a7f3e21 [2026-05-12] CC — rewrite hero copy  
CI run: success  
Build: bun run build → exit 0  
Cleanup: worktree removed, branch deleted
```

Six lines. Every one of them a fact you can independently check. No prose, no narration, no “everything looks good.” This is the format you want every lane returning to the main thread.

WHERE THE RULES LIVE

These five rules belong in your `CLAUDE.md` so every lane reads them before starting. If you’re using the Dev-Team Document Set kit, the rules are baked into the templates. If you’re rolling your own, copy them verbatim — they’re cheap to include and expensive to skip.

SECTION 6.

How to write a clean lane prompt

A lane prompt is not a wish. It's a contract. The cleaner the contract, the less you have to babysit, and the less surface area there is for a lane to misread what you wanted and ship something you didn't.

I write lane prompts the same way every time, in the same order, because the order is what keeps me from skipping the parts I always want to skip. Five parts, ten minutes to write the first one, two minutes once you have a template.

Verbatim copy beats paraphrase. The temptation is to "translate" your intent into agent-friendly language — turn "make the hero copy say what's actually true" into "Update the H1 element to reflect a more authentic value proposition." Don't. The original sentence is closer to what you mean than the translation is. The lane reads your real voice and stays closer to your real intent.

Success criteria: name them as observable facts. Not "make sure it works." The criteria you'd run yourself to check that the lane did the thing. For a code lane, that's usually three: a commit SHA on `origin/main`, a grep that the change is visible in the source on main, and a behavior check — a curl against the deployed URL with a cache-buster, or a screenshot, or a build log.

A clean criteria block reads like this:

Success means all three:

1. commit SHA on origin/main (``git log origin/main -1 --format='%h %s'``)
 2. ``grep -r "AI Pocket Agency" src/`` returns the new copy from origin/main
 3. ``curl -sI "https://aipocketagency.com/?_=$(date +%s)"`` returns 200
- AND the rendered HTML contains the new line.

Three commands. Three pass/fail. The lane runs them, copies the output into its report, and you read four lines instead of four paragraphs to know whether the thing shipped.

Blockers: tell the lane to stop and report, not to guess. Every nontrivial lane hits an edge case. A package version disagrees with the lockfile. A migration file already exists at the intended timestamp. A clean prompt names the most likely edge cases up front and tells the lane explicitly: *if you hit this, stop and report, do not improvise.*

Standing rules: link to the brain, don't restate. Every lane reads `CLAUDE.md` and `AGENTS.md` at session start. Those files already contain the rules from section 5. Drop a one-liner that points at the relevant brain memory entry — "Follow `feedback_verify_push_to_main.md`" — and trust the standing files.

Tool tips: name the local potholes by name. When the lane needs to run `bun install` and the sandbox is FUSE-mounted, tell it to use Desktop Commander instead. When the lane needs a GitHub token, tell it where the token comes from. Don't make the lane figure out the auth dance from scratch on every run. Bake the answer into the prompt.

A complete lane prompt fits in one screen. The opening line is the verbatim ask. The next block is the success criteria. The next block is the blockers and the "stop and report" trigger. The next line is the standing-rules pointer. The last block is the tool tips. Done.

SECTION 7.

Verification discipline: four layers of done

There are four layers of “is this actually done,” and conflating them is the single most expensive mistake you can make as an orchestrator.

The brain memory entries `feedback_verify_before_asserting.md` and `feedback_verify_push_to_main.md` exist because I learned this one the hard way — twelve lanes reporting “shipped clean” while GitHub Actions CI was red on every single one of them for about four hours, because nobody in the chain ran `gh run list` before relaying the self-reports.

Layer one — lane self-report. “Done. Pushed. All green.” It is the least trustworthy layer. It might be true. It often is true. But the lane is grading its own paper. Treat lane self-reports as the *starting point* for verification, not the end of it.

Layer two — disk-verified. The file exists at the expected path. The content matches what the lane said it wrote. `grep` finds the new string. This is cheap to check and catches the most embarrassing failures — the lane that “wrote the file” to a path that doesn’t exist, the lane that edited the wrong file in the wrong worktree.

Layer three — remote-verified. The commit is on `origin/main`. The SHA matches `git ls-remote origin main`. The CI run is green. This is the layer where most “shipped” claims live or die.

Concrete verification snippets you’ll run constantly:

```
# What's the tip of origin/main right now?
git fetch origin && git log origin/main -1 --format='%h %s'

# Did the latest CI run pass?
gh run list --limit 5

# What's the live response code, cache-busted?
curl -sI "https://aipocketagency.com/?_=$(date +%s)" | head -1

# Does the rendered HTML actually contain the new copy?
curl -s "https://aipocketagency.com/?_=$(date +%s)" | grep -i "ai pocket agency"
```

Four commands. Maybe ten seconds total. If you skip them, you ship with your eyes closed.

Layer four — behavior-verified. The live URL returns the expected response. The newly-deployed edge function answers a real request with the new schema. The user-facing flow you changed actually does the new thing when you click through it. This is the hardest layer to automate, the easiest to skip, and the one that catches the bugs that none of the lower layers can see.

The rule I follow when reporting up to a user: **specify which layer each claim is at.** Don't say "shipped" when you mean "remote-verified." Don't say "verified" when you mean "lane-reported." It feels verbose in writing. It's three seconds to say out loud. And it's the difference between "the orchestrator told me five things shipped and three of them didn't" and "the orchestrator told me five things shipped at the layer I cared about, and they did."

SENSITIVE ENV-VAR GOTCHA

Sensitive Vercel env vars cannot be value-verified via the API. Diagnostics can confirm a sensitive env var EXISTS, never its VALUE. To verify the value, you either curl the third party with the key, or add a temporary runtime probe, or exercise the actual code path and watch the logs. Don't let a layer-three diagnostic mislead you about something that requires layer-four verification.

SECTION 8.

Parallel orchestration patterns

The mechanics of parallel lanes are simple. The patterns for *when* to run lanes in parallel and how they relate to each other are where the orchestration craft lives. Three patterns cover almost every multi-lane day.

Sequential dependency. Lane B can't start until Lane A pushes its commit. Maybe Lane A is creating a migration that Lane B needs to read from. Maybe Lane A is regenerating types that Lane B will import. Whatever the reason, the rule is: don't spawn Lane B until Lane A's SHA is on `origin/main`. The cost of doing this wrong is Lane B starting against a stale base, pushing, hitting a rebase conflict, and you having to babysit a merge you didn't need to have in the first place.

Fan-out. N lanes that don't touch each other's files. This is the pattern you want. Fire all of them at once. Each one rebases on push, lands sequentially on the remote because git serializes pushes to main, no conflicts because the files are disjoint. Four lanes that each take fifteen minutes serialized is an hour. Four lanes fanned out is fifteen minutes. The thing that makes fan-out work is the file disjointness check — before you spawn the lanes, look at the files each one is going to touch.

Watch-and-merge. One lane builds while another verifies. This is the pattern for when a build takes time and you don't want to block the main thread waiting on it. Lane A is doing the long build. Lane B, in the meantime, is running the verification suite for the previous deploy. The main thread is the coordinator. It receives both reports independently, and decides whether to ship the new commit or roll back.

When NOT to parallelize. The honest answer is "more often than you think." Lanes that touch the same file or the same lines of the same file should be sequenced, not parallelized. Lanes that share a generated artifact — `types.ts`, `bun.lockb`, a regenerated schema file — should have one owner for that artifact, not multiple. Tightly coupled refactors that ripple across the whole codebase should usually be one lane, because the next step depends on the previous step in a way that doesn't decompose cleanly.

The discipline is: **name the pattern before you spawn anything.** "This is a three-lane fan-out, no shared files." "This is a two-step sequential dependency — Lane A's SHA gates Lane B's spawn." "This is watch-and-merge — Lane A builds, Lane B audits the current state." Once you've named it, you know what verification rhythm to follow and what to watch for.

SECTION 9.

Common failure modes

These are the failure modes I hit often enough that they have names. None of them are exotic. The fastest way through them is recognition — when something looks like one of these, you treat it like one of these.

Duplicate lanes from dispatch timeouts. The `start_code_task` tool can return a 180-second timeout while the session it was spawning is still alive on the host. Symptom: an error that says the task didn't start. Reality: it did start, you just don't have the handle. If you spawn again to "retry," you now have two lanes running the same prompt. The fix: after any `start_code_task` timeout, call `list_sessions` to see what's actually running. If two sessions have the same prompt and target, kill the duplicate via `send_message` with a STOP instruction before either pushes.

Stale branches. Worktree branches never auto-clean unless you tell them to. Pile up across all repos and you wake up next week with forty-plus stale `task/<name>` branches. The rule from section 5 — every lane cleans up after itself on a verified push — is the primary defense. The scheduled cleanup task that sweeps abandoned branches every six hours is the backstop. Skip them and the noise eats you.

Push conflicts. Non-fast-forward rejected when `origin/main` moved between the time the lane started and the time the lane tried to push. The fix is rebase plus retry: `git fetch origin && git rebase origin/main && git push origin HEAD:main`. The lane already does the rebase at start, per rule 1, so this is only an issue if main moved *during* the lane's work.

FUSE EPERM on `bun install` or `bun run build`. The workspace bash that comes with most sandboxed environments mounts the project via FUSE, and FUSE blocks `unlink()` for `node_modules` and the temp scratch directories modern bundlers create. Symptom: `EPERM unlink` errors halfway through `bun install`. The fix is to run those commands via Desktop Commander's `mcp__Desktop_Commander__start_process`, which operates on the host filesystem directly without the FUSE shim. The shorthand: anything that exercises a toolchain's temp-file thrash needs Desktop Commander; anything that's a simple file read or git command is fine in workspace bash.

Token file doesn't exist. A prompt that hardcodes `~/.config/github/token` will work on the host where that file exists and fail silently on every other host. The fallback that always works when `gh` is authenticated is the `osxkeychain` credential helper — git is already configured to ask it for the token at push time. Or fall back to the 1Password CLI via the `getsecret` helper. The pattern: don't hardcode a token path unless you've verified that path exists on the specific host the lane will run on.

Silent catches eating errors. Every `catch` block must call `console.error` or `Sentry.captureException` at minimum. Empty `catch {}` and `.catch(() => {})` patterns are forbidden because they cause silent-exit bugs that take hours to diagnose. The canonical incident — a Calendly webhook handler that silently returned 200 on every real delivery for months because a missing field tripped an early-return guard that nobody could see.

Webhook handlers built from documentation instead of actual payloads. Whenever you're integrating a third-party webhook for the first time, deploy with a temporary `console.log('[webhook-name] body:', JSON.stringify(body))` at the top of the route, trigger one real delivery, inspect the structure in your platform logs, and only then write the handler logic against the real wire shape. Synthetic test payloads tend to match whatever the handler expects, so they verify nothing.

SECTION 10.

Real example walkthroughs

Three scenarios from real Whited Consulting work in the last few weeks. Each one is a complete arc — what was being shipped, what the orchestrator did, what went right, what almost went wrong, and the rule the scenario reinforces.

Scenario A — Multi-lane website rewrite

The brief was four edits on the same Next.js marketing site. New hero copy. A broken footer URL. A removed Twitter link. A monospace marker added to the manifesto headline. Single-thread version of that day would have been about three hours of context-swapping. Dispatch version, the goal was thirty minutes.

The orchestrator spawned four code tasks at once. Each one had its own worktree on its own task branch. Each prompt included the section-5 standing rules as a preamble. Each prompt named the one file the lane was responsible for and the files it was forbidden from touching. Disjoint scopes by hard rule.

All four lanes reported back within twenty minutes. Each report had the format from section 5 rule 5 — branch, commit SHA, origin/main tip, CI status, build exit code, cleanup status. The orchestrator independently verified each SHA with `git ls-remote origin main`, ran a cache-busted curl against the deployed URL, and only then promoted the lane's claim from "lane-reported" to "behavior-verified."

What almost went wrong: one of the four lanes pushed its commit but didn't run the build step before pushing. The build was actually fine — a single-line copy edit — but the report came back without the `bun run build → exit 0` line, which means the lane skipped a rule. The orchestrator flagged it as a layer-three verification instead of layer-four. The rule got reinforced in the next prompt template. **Skipped steps that don't bite you the first time still get flagged.**

Scenario B — Brain doc bundle with rebases

Two lanes both writing markdown to `whited-brain` at the same time. Lane A was logging an intel document about a competitor. Lane B was writing a product spec for a new APA offering. Different files, different paths, no shared content. A clean fan-out — two lanes, no overlap, fire at once.

Both lanes spawned within seconds of each other. Both did the section-5 rule-1 rebase at start, came up clean. Lane A finished first because it was a shorter doc — pushed its commit

to `origin/main`, reported its SHA, cleaned up. Lane B finished about ten minutes later, ran its rebase against the *new* tip of `origin/main` (which now contained Lane A's commit), still came up clean, pushed its own commit, cleaned up.

The orchestrator verified both SHAs independently against `git ls-remote origin main`, confirmed both files were present in the right paths with `git show origin/main:path/to/file | head -20`, and reported back. Two commits, two verified SHAs, two cleanly-cleaned-up worktrees, fifteen minutes elapsed.

Rule reinforced: **clean fan-outs in a doc repo work the same way as clean fan-outs in a code repo**. The pattern doesn't care about the file type. It cares about file disjointness and the rebase rhythm.

Scenario C — Cleanup after dispatch timeout zombies

Two `start_code_task` calls in the same session both timed out at 180 seconds on the orchestrator's end. Both then silently spawned real sessions on the host. The orchestrator, not knowing this, retried both — and now there were four sessions running the same two prompts.

The first sign something was wrong was a `list_sessions` call about ten minutes in that returned three active sessions where the orchestrator expected one. The orchestrator immediately stopped and audited. Three sessions, two distinct prompts, one prompt represented by two sessions. The duplicate had not yet pushed any commit but had completed its work and was about to.

The orchestrator sent a STOP message to the duplicate: "STOP. Do not push. Run `git checkout -- . && git clean -fd` and idle clean." The duplicate complied. The primary pushed cleanly, the orchestrator verified the SHA, and the operation finished with no garbage in main.

Rule reinforced: **after any dispatch timeout, run `list_sessions` before doing anything else**. The mitigation is cheap, the failure is expensive, and the pattern is exactly what fast recognition is for.

SECTION 11.

What to do when it goes wrong

Things will go wrong. The difference between an orchestrator who handles parallel work cleanly and one who doesn't is not that the first one never breaks things. It's that the first one has a clear recovery sequence for each failure mode and applies it without panicking.

Abort a lane mid-flight. If a lane is doing the wrong thing — wrong files, wrong scope, wrong approach — `send_message` it a STOP with explicit revert instructions:

```
STOP. Do not commit anything. Run:
  git checkout -- .
  git clean -fd
Then idle clean and report when reverted.
```

The lane runs those two commands, drops every uncommitted change in the worktree, and stops. The worktree is back to clean. The branch hasn't moved. Nothing has been pushed. You've spent thirty seconds and lost nothing.

Roll back a bad commit on main. If a commit is already on `origin/main` and you need to undo it, the move is `git revert <sha>` from a fresh worktree, not a force-push. Revert creates a new commit that undoes the bad one. The history stays linear and visible. Force-pushing main to remove the bad commit is destructive: it rewrites history, it breaks anyone who already pulled the bad SHA, and it makes the next person to look at the repo wonder why the timestamps don't line up. Use `git revert`. Verify the revert deployed by curling the live URL with a cache-buster.

Recover from a divergent worktree. Sometimes a worktree gets tangled enough that rebasing it back to `origin/main` keeps hitting the same conflict and the lane can't make progress. The recovery sequence:

```
# In the tangled worktree
git stash push -m "rescue-divergent-work"
git fetch origin
git reset --hard origin/main
# Decide whether the stashed work is worth re-applying
git stash show -p stash@{0} | less
# If useful: `git stash apply` and resolve.
# If not: `git stash drop stash@{0}`.
```

The stash is the parachute. Reset hard to `origin/main`, then decide whether the stashed work is worth re-applying.

Lost work in a stash conflict. A stash conflict produces merge markers when the stash is applied on top of a base it doesn't fit. The hard rule: when a stash conflict shows up and the stash's intent is destructive — overwriting recent work, reverting to an older state — restore from HEAD and drop. When the stash's intent is constructive — adding work that should be preserved — resolve the conflict by hand. The first move is reading the stash before deciding.

API error mid-turn. Sometimes the orchestrator hits an API error mid-turn. The state of the world doesn't disappear: the lane that was running is still running, the commits that were on `origin/main` are still there. The recovery is to resume the same session with `send_message` summarizing what was discovered before the error. State is preserved on the host. The world's state is the source of truth. Your memory of the world's state is a guess.

When you've made a mistake you can't undo cleanly. This part isn't a recipe, it's a posture. If a lane pushed something irreversible — a deletion that propagated, a config change that broke a downstream system, a deploy that took something offline that other people are using — the move is to stop, surface the situation clearly to whoever needs to know, and ask. Don't compound the mistake with a hasty fix. Don't try to make the problem invisible by reverting silently. State what happened, state what the current world looks like, state what your proposed recovery is, ask for confirmation. The cost of a thirty-second pause to confirm is bounded. The cost of an autonomous "fix" that turns out to be wrong is unbounded.

THE BAR

If you can read these eleven sections and still feel scared, run a fan-out of three lanes on something low-stakes — rename a variable across three files in a small repo, or write three small markdown notes in a doc repo — and watch them work. Confidence here is hands-on, not theoretical. The first clean four-lane day is the day you stop being scared, and there is no shortcut to it. The playbook is the map. The clean four-lane day is the territory.

Up next: the kit, the community, the dashboard. The Dispatch Playbook is the first product in a larger line. The Dev-Team Document Set ships the `CLAUDE.md`, `AGENTS.md`, brain-memory templates, and worktree-cleanup scheduled task as ready-to-drop files in any repo. The AI Pocket Agency community at skool.com/aipocketagency is where the live orchestrators trade pattern updates as the tooling evolves. Each of those builds on the orchestration pattern this playbook lays out. None of them works without it.